# 天 津 理 工 大 学

# 实 验 报 告

## 实验内容

学院（系）名称：华信软件学院

### 一.实验报告评分表（教师用）

| 实验名称 | MyBatis 与 MyBatis-Plus 环境搭建 | 姓名 | 赵亮 |
|---|---|---|---|
| 学号 | 20241850 | 班级 | 4 班 |
| 日期 | 2026.10.12 | 指导教师 | 庄凯 |

| 评价维度 | 满分 | 评分要点 | 得分 |
|---|---|---|---|
| 环境搭建与功能开发 | 40 | 环境完整、功能正确、可运行 | |
| 问题排查能力 | 20 | 独立定位并解决报错，记录≥2 问题 | |
| AI 工具规范化应用 | 20 | 保留记录、人工校验、隐私合规 | |
| 实验报告质量 | 20 | 六部分完整规范真实 | |
| 合计 | 100 | 百分制 S = 各维度得分之和 | |

百分制得分 S：_____ ｜ 折算课程实验分（10 分制，= S ÷ 100 × 10 × 学时/16）：_____

教 师 评 语 ：

_____

_____

教师签字：_____ 日期：_____

| 姓名 | 赵亮 | 学号 | 20241850 | 专业 | 软件工程 |
|---|---|---|---|---|---|
| 班级 | 4 班 | 实验项目 | 实验一：MyBatis 与 MyBatis-Plus 环境搭建及 CRUD 实操 | 实验时间 | 2026.10.12 |
| 课程名称 | Java 框架技术（一） | 课程代码 | 1960229 | 实验地点 | B409 |

批改意见：

_____

成绩：_____ 教师签字：_____ 日期：_____

## 二. 实验环境

| 类别 | 软件 / 组件 | 版本 | 说明 |
|---|---|---|---|
| 操作系统 | macOS | 26.3.1（arm64） | 开发主机，Apple 芯片 |
| JDK | OpenJDK 17 | 17.0.19 | 编译级别 release 17 |
| 构建工具 | Apache Maven | 3.10.0 | 已配置阿里云镜像加速依赖下载 |
| 数据库 | MySQL Server | 9.6.0 | 本地实例，库名 ssm_emp（字符集 utf8mb4） |
| JDBC 驱动 | mysql-connector-j | 9.0.0 | 与 MySQL 9.x 同代的新坐标（com.mysql:mysql-connector-j） |
| 持久层框架 | MyBatis | 3.5.16 | 原生轨（XML 映射 + 注解映射） |
| 增强框架 | MyBatis-Plus | 3.5.5 | 通用 CRUD / 条件构造器 / 分页插件 |
| 日志组件 | Log4j | 1.2.17 | logImpl=LOG4J，用于输出 SQL 日志 |
| 单元测试 | JUnit | 4.13.2 | 15 个测试类、116 个测试用例 |
| 辅助库 | Lombok | 1.18.48 | 生成 getter/setter；该版本与 JDK 17 编译器兼容 |
| 版本控制 | Git | 2.55.0 | 按功能分批提交，便于回溯与复盘 |
| 集成开发环境 | IntelliJ IDEA | 2026.1.1 | — |
| AI 编程插件 | DeepSeek 编程助手 | — | 用于代码生成与报错排查 |

## 三. 实验目标

4.使用 AI 编程插件辅助，从零搭建 MyBatis 与 MyBatis-Plus 工程环境，掌握 Maven 依赖管理与配置报错的排查方法；

5.掌握 MyBatis 基础 CRUD 开发，理解 SqlSessionFactory、SqlSession、Mapper 接口三者的关系与生命周期；

6.掌握动态 SQL（<if>/<where>/<trim>/<set>/<choose>/<foreach>）、多条件查询、批量操作与分页查询的实现；

7.掌握 MyBatis-Plus 的通用 CRUD（BaseMapper）、条件构造器（QueryWrapper / LambdaQueryWrapper）与分页插件的配置与使用；

8.集成 Log4j 日志与 JUnit 单元测试，能够依据 SQL 日志排查参数绑定与结果映射问题；

9.培养利用 AI 辅助排查框架配置异常、并对 AI 生成代码进行人工校验的能力。

# 四、实验内容与过程

## 3.1 数据库设计

在 emp 表基础上，新增了若干字段与一张部门维表，目的是让后续的“动态 SQL、多条件查询、统计与 N+1”有真实数据可依：

| 表 | 字段 | 关键设计 |
| --- | --- | --- |
| emp | 17 | 保留 emp_id / emp_name / gender / dept / post / salary / hire_date / status 等基础字段，新增 emp_no（业务唯一键）、birth_date、email、phone、dept_id（外键）、del_flag（逻辑删除）、version、create_time、update_time |
| sys_dept | 8 | 部门维表：dept_id / dept_code / dept_name / location / manager / budget / create_time / update_time |

其中最重要的一处设计是把“业务状态”与“逻辑删除”拆成两个字段：

- status：业务状态，1 在职 / 0 离职；
- del_flag：逻辑删除标记，0 正常 / 1 已删除（MyBatis-Plus 的 @TableLogic 专用）。

如果把 @TableLogic 直接标注在表示业务状态的 status 上，MyBatis-Plus 会把“离职”当成“已删除”，查询条件与删除语义会整体反转（详见第五节问题 3），因此这里专门用独立字段承载。

种子数据共 6 个部门 + 16 名员工，刻意覆盖边界：在职 12 条、离职 3 条、已逻辑删除 1 条；email / phone / birth_date 各留空值；薪资区间 4500 ~ 32000；入职年份 2015 ~ 2025 ——这些为后面的分页、区间筛选与统计提供了数据基础。

建库、导入与基线复核命令：

```
mysql -u root -p < sql/01_schema.sql     # 建库建表
mysql -u root -p < sql/02_seed.sql       # 导入种子数据
mysql -u root -p < sql/03_verify.sql     # 复核基线：16 / 15 / 1 / 12 / 3
```

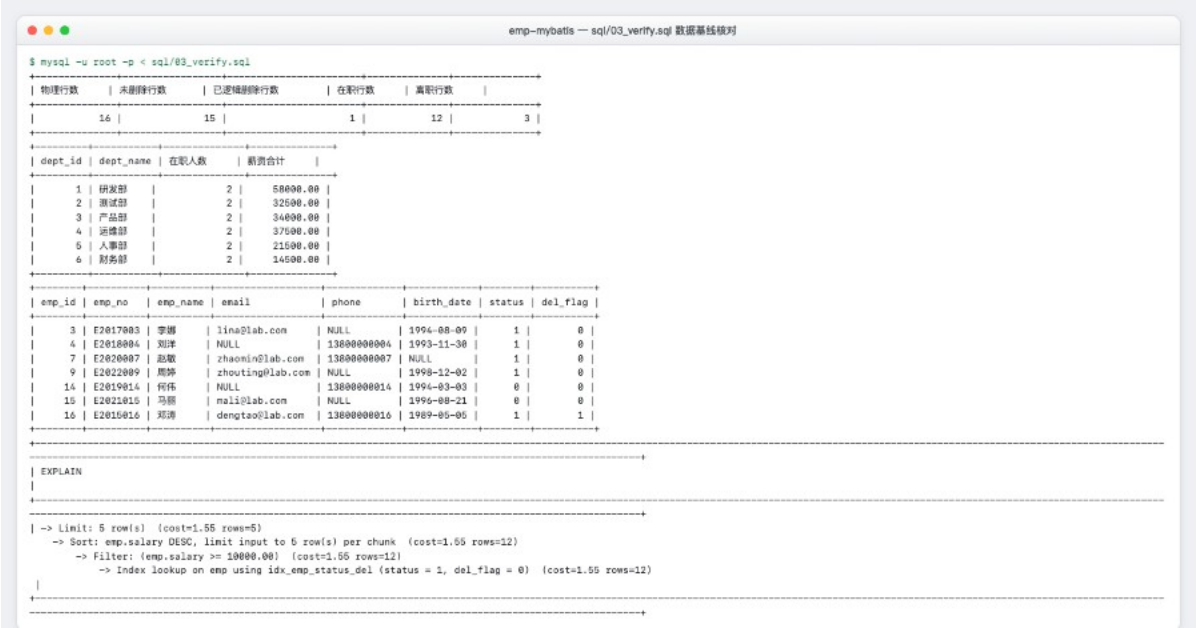



图 3-1　数据基线核对结果

## 3.2 工程结构

工程取名 emp-mybatis，用单模块的 Maven 工程组织，一个 pom.xml 管全部依赖：

```
emp-mybatis/
├── pom.xml                          单模块工程（groupId: com.lab）
├── sql/                             01 建表 / 02 种子数据 / 03 基线核对
├── scripts/                         启动 MySQL、一键测试、生成报告
├── demo.sh                          答辩演示脚本
└── src/
    ├── main/java/com/lab/
    │   ├── LabApplication.java      可运行入口
    │   ├── common/                  实体 / DTO / 工具 / 异常
    │   ├── mybatis/                 原生 MyBatis：会话工厂、Mapper、慢 SQL 插件、Service
    │   └── plus/                    MyBatis-Plus：配置工厂、BaseMapper
    ├── main/resources/              mybatis-config.xml、log4j、映射文件
    └── test/java/com/lab/           15 个测试类 / 116 个用例
```

单模块的好处是结构简单、老师一看就懂；两条技术路线靠包名分开（mybatis 与 plus），依赖都在同一个 pom.xml 里，版本也不用跨模块仲裁。

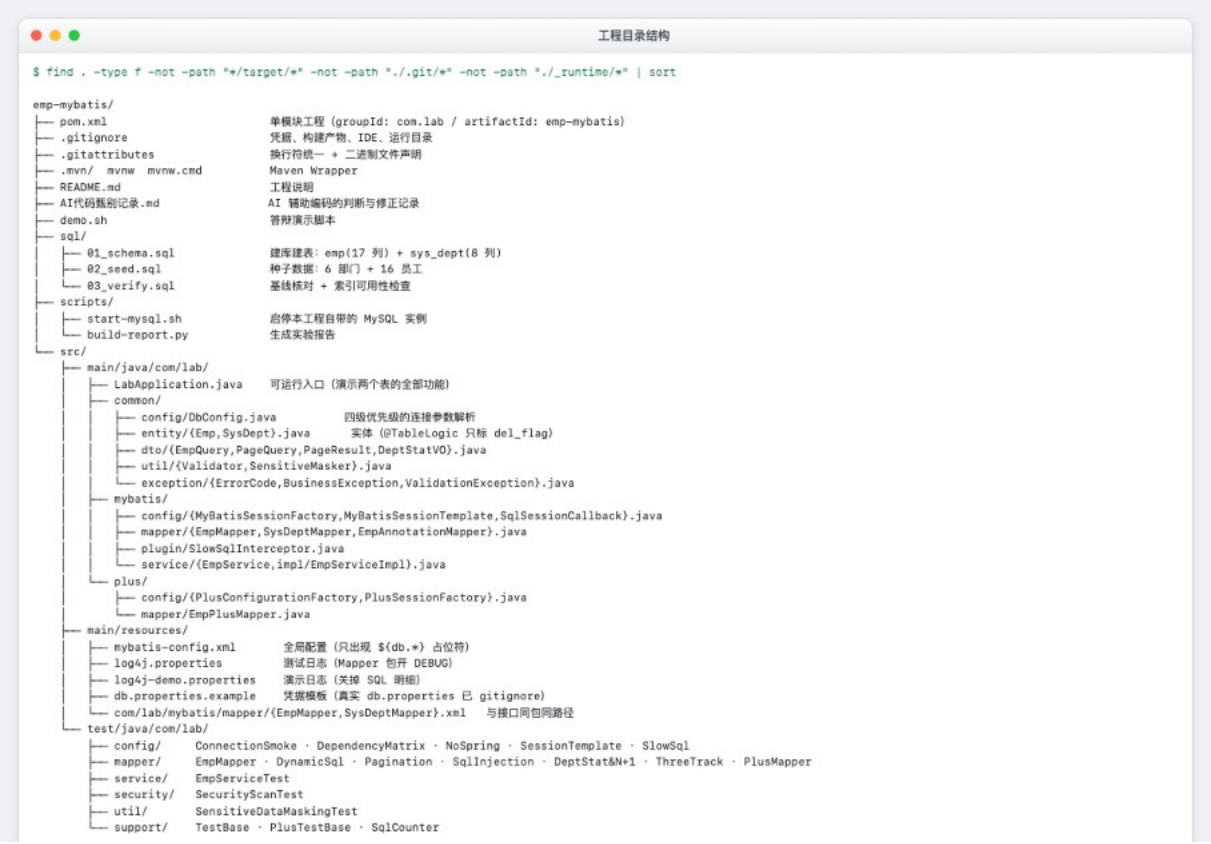



图 3-2 工程目录结构

## 3.3 任务 1：环境搭建与配置

（1）数据库连接信息不写进代码。连接参数按“-D 系统属性 > 环境变量 > db.properties > 非敏感默认值”四级优先级解析；口令没有默认值，未配置时直接抛出明确异常，避免“悄悄用空口令连库”：

```
public static DbConfig load() {
    Properties file = loadOptionalFile();            // classpath:db.properties（可选，不入库）
    String driver   = resolve("db.driver",   "DB_DRIVER",   DEFAULT_DRIVER,   file);
    String url      = resolve("db.url",      "DB_URL",      DEFAULT_URL,      file);
    String username = resolve("db.username", "DB_USERNAME", DEFAULT_USERNAME, file);
    String password = resolveSecret(file);           // 无默认值，缺失即抛异常
    return new DbConfig(driver, url, username, password);
}
```

配置文件 mybatis-config.xml 中只出现占位符，真实口令由运行时注入：

```
<properties>
    <property name="driver"   value="${db.driver}"/>
    <property name="url"      value="${db.url}"/>
    <property name="username" value="${db.username}"/>
    <property name="password" value="${db.password}"/>
</properties>
```

```
<settings>
    <setting name="mapUnderscoreToCamelCase" value="true"/>
    <setting name="logImpl" value="LOG4J"/>
    <setting name="defaultStatementTimeout" value="10"/>
</settings>
```

（2）Mapper 映射文件的存放位置。<mappers> 采用 <package> 扫描接口，这种扫描方式下 MyBatis 只按“接口全限定名把 . 换成 / 再加 .xml”去找映射文件，所以映射文件必须与接口同包同路径存放（src/main/resources/com/lab/mybatis/mapper/EmpMapper.xml）。

（3）日志配置。log4j.properties 只对 Mapper 包开 DEBUG，以便看到 Preparing / Parameters / Total / Updates 四类关键行，同时抑制框架自身的噪音输出。

（4）环境自检。编写了三个不依赖业务数据的测试类：连接冒烟（ConnectionSmokeTest）、依赖矩阵（DependencyMatrixTest）、以及依赖与约束校验（NoSpringDependencyTest），保证环境问题能在第一步就被发现。

### 3.4 任务 2：MyBatis 基础 CRUD

（1）Mapper 接口与映射文件。接口只声明方法，SQL 全部写在 XML 中；映射文件用显式 <resultMap> 声明“列名 → 属性名”，不依赖全局驼峰开关：

```
<resultMap id="empMap" type="Emp">
    <id     property="empId"    column="emp_id"/>
    <result property="empNo"    column="emp_no"/>
    <result property="empName"  column="emp_name"/>
    <result property="hireDate" column="hire_date"/>
    <result property="delFlag"  column="del_flag"/>
    <!-- 其余字段略 -->
</resultMap>
```

新增时用 useGeneratedKeys 回填自增主键，这是后续“插入后立刻用它查询”的前提：

```
<insert id="insert" parameterType="Emp" useGeneratedKeys="true" keyProperty="empId">
    INSERT INTO emp (emp_no, emp_name, gender, ..., status, del_flag, version)
    VALUES (#{empNo}, #{empName}, #{gender}, ..., #{status}, 0, 0)
</insert>
```

（2）会话与事务边界。原生方式需要自己开关会话并提交事务，容易漏写 commit()（漏写时数据会在会话关闭时被回滚，表现为“程序说成功了、数据库里却没有”）。为此封装了一个函数式会话模板，把“开关、提交、回滚、关闭”收敛到一处：

```
public <T> T withSession(SqlSessionCallback<T> callback) {
    SqlSession session = sessionSupplier.get();
    try {
        T result = callback.apply(session);
        session.commit();
        return result;
    } catch (RuntimeException | Error e) {
```

```
            session.rollback();
            throw e;
        } finally {
            session.close();
        }
    }
```

业务层因此只表达“做什么”，不再关心事务：

```
    @Override
    public Integer save(final Emp emp) {
        Validator.validateForInsert(emp);          // 校验放在事务外，参数不合法时不必开连接
        return template.withSession(session -> {
            EmpMapper mapper = session.getMapper(EmpMapper.class);
            mapper.insert(emp);
            return emp.getEmpId();                  // useGeneratedKeys 回填后的主键
        });
    }
```

（3）单元测试。测试基类为每个测试方法开一个 autocommit=false 的会话，结束时统一 rollback()，保证测试不污染数据库基线：

```
    @Before public void openSession() {
        purgeLeftoverTestRows();                    // 兜底清理中断遗留的数据
        session = MyBatisSessionFactory.openSession();
        mapper  = session.getMapper(EmpMapper.class);
    }
    @After  public void rollbackAndClose() {
        if (session != null) { session.rollback(); session.close(); }
    }
```

## 3.5 任务 3：动态 SQL、多条件查询与批量操作

（1）公共条件片段 + 两种容器。把各条 <if> 条件抽成独立片段，再用 <where> 与 <trim> 两种容器分别包裹，用于对比两者的等价性：

```
<sql id="conditionItems">
    <if test="q.onlyAlive == null or q.onlyAlive == true">AND del_flag = 0</if>
    <if test="q.empName != null and q.empName != ''">
        AND emp_name LIKE CONCAT('%', #{q.empName}, '%')
    </if>
    <if test="q.deptId != null">AND dept_id = #{q.deptId}</if>
    <if test="q.minSalary != null">AND salary &gt;= #{q.minSalary}</if>
    <if test="q.hireDateFrom != null">AND hire_date &gt;= #{q.hireDateFrom}</if>
</sql>

<sql id="conditionWhere"><where><include refid="conditionItems"/></where></sql>
<sql id="conditionWhereTrim">
    <trim prefix="WHERE" prefixOverrides="AND |OR "><include
refid="conditionItems"/></trim>
</sql>
```

多条件查询与计数共用同一片段，从而保证“列表”与“分页总数”口径完全一致；排序字段用 ${} 拼接（动态列名只能这样），但取值经过白名单校验：

```
ORDER BY ${q.safeOrderBy}
<choose><when test="q.ascending">ASC</when><otherwise>DESC</otherwise></choose>
<if test="page != null">LIMIT #{page.offset}, #{page.safePageSize}</if>
```

（2）动态更新（<set>）与批量操作（<foreach>）。<set> 会自动去掉多余的尾逗号，只更新非空字段：

```
<update id="updateDynamic">
    UPDATE emp
    <set>
        <if test="emp.empName != null and emp.empName != ''">emp_name =
#{emp.empName},</if>
        <if test="emp.post != null and emp.post != ''">post = #{emp.post},</if>
        <if test="emp.salary != null">salary = #{emp.salary},</if>
        version = version + 1
    </set>
    WHERE emp_id = #{emp.empId}
</update>
```

批量插入用一条多行 VALUES 语句完成，useGeneratedKeys 会逐行回填主键：

```
<insert id="insertBatch" useGeneratedKeys="true" keyProperty="empId">
    INSERT INTO emp (emp_no, emp_name, gender, dept_id, dept, post, salary,
hire_date, status, del_flag, version)
    VALUES
    <foreach collection="list" item="e" separator=",">
        (#{e.empNo}, #{e.empName}, #{e.gender}, #{e.deptId}, #{e.dept},
         #{e.post}, #{e.salary}, #{e.hireDate}, #{e.status}, 0, 0)
    </foreach>
</insert>
```

（3）分支选择（<choose>）。实现“优先按姓名精确查 → 否则按部门查 → 否则兜底查在职”的三级分支。

（4）分页。原生轨采用“两条 SQL”的自研分页：先用同一套条件执行 COUNT 得到总数，再取当前页数据，由 PageResult 承载并计算总页数；PageQuery 对页码与页大小做归一化（页码 ≥ 1，页大小 1~100），避免“一次查全表”。

## 3.6 任务 4：MyBatis-Plus 通用 CRUD 与分页

（1）手工装配 MyBatis-Plus 运行环境。这次实验没用任何 starter，而是直接手工装配运行环境：数据源、驼峰映射、日志实现、Mapper 注册与插件注册全部在代码里完成。其中最容易被忽略的一点是配置类的类型必须用 MybatisConfiguration（详见第五节问题 1）：

```
public static MybatisConfiguration buildConfiguration() {
    DbConfig db = DbConfig.load();
    MybatisConfiguration configuration = new MybatisConfiguration();   // 关键点
```

```
    configuration.setMapUnderscoreToCamelCase(true);
    configuration.setLogImpl(Log4jImpl.class);
    configuration.setEnvironment(new Environment(ENVIRONMENT_ID,
            new JdbcTransactionFactory(), new PooledDataSource(
                    db.getDriver(), db.getUrl(), db.getUsername(),
db.getPassword())));
    configuration.addInterceptor(mybatisPlusInterceptor());
    configuration.addMapper(EmpPlusMapper.class);
    return configuration;
}

public static SqlSessionFactory build() {
    return new MybatisSqlSessionFactoryBuilder().build(buildConfiguration());
}
```

（2）零 SQL 的通用 CRUD。实体上标注 @TableName/@TableId/@TableLogic/@Version 后，Mapper 只需继承 BaseMapper<Emp> 就获得完整 CRUD 能力。实测自动注入 12 条语句：insert / delete / deleteById / deleteBatchIds / update / updateById / selectById / selectBatchIds / selectList / selectMaps / selectObjs / selectCount，再加上自己写的 3 条 @Select，共 15 条 MappedStatement。

（3）条件构造器。用 LambdaQueryWrapper 声明条件，避免手写列名字符串：

```
LambdaQueryWrapper<Emp> wrapper = Wrappers.<Emp>lambdaQuery()
        .eq(Emp::getDeptId, 1)
        .eq(Emp::getStatus, 1)
        .orderByAsc(Emp::getEmpId);
List<Emp> list = empPlusMapper.selectList(wrapper);
```

（4）分页插件。注册 MybatisPlusInterceptor 并加入 PaginationInnerInterceptor(DbType.MYSQL) 后，selectPage 会自动执行一次 COUNT 查询并在原 SQL 上追加 LIMIT：

```
Page<Emp> firstPage = empPlusMapper.selectPage(new Page<>(1, 4),
        Wrappers.<Emp>lambdaQuery().orderByAsc(Emp::getEmpId));
// firstPage.getTotal() = 15, getPages() = 4, getRecords().size() = 4
```

（5）逻辑删除。del_flag 上标注 @TableLogic(value = "0", delval = "1") 后，MyBatis-Plus 会自动为查询追加 del_flag = 0 条件，而 deleteById 执行的是 UPDATE emp SET del_flag = 1（逻辑删除）而非物理 DELETE。

## 3.7 任务 5：日志排查、代码审查与安全加固

（1）慢 SQL 插件。手写一个 MyBatis Interceptor 统计并告警慢 SQL。拦截点的选择有讲究：prepare() 只负责“准备语句”，真正的执行发生在 query()/update() 里，统计耗时必须拦后者：

```
@Intercepts({
    @Signature(type = StatementHandler.class, method = "query",
               args = {Statement.class, ResultHandler.class}),
```

```
    @Signature(type = StatementHandler.class, method = "update",
               args = {Statement.class})
})
public class SlowSqlInterceptor implements Interceptor {
    @Override
    public Object intercept(Invocation invocation) throws Throwable {
        long startNanos = System.nanoTime();
        HIT_COUNT.incrementAndGet();
        try {
            return invocation.proceed();
        } finally {
            long costMs = (System.nanoTime() - startNanos) / 1_000_000L;
            if (costMs >= resolveThreshold()) {
                LOG.warn("[SLOW-SQL] " + costMs + " ms | " + safeSql(invocation));
            }
        }
    }
}
```

阈值支持 -DslowSql.thresholdMs > 环境变量 SLOW_SQL_THRESHOLD_MS > 配置文件 > 默认 200ms 四级回退；插件在 mybatis-config.xml 的 <plugins> 段注册。实测：用 -DslowSql.thresholdMs=0 打开全量告警后，每条语句都会输出真实 SQL 文本与耗时。




图 3-3　阈值设为 0 时的慢 SQL 输出

（2）注解映射轨与三轨一致性。除 XML 轨与 MyBatis-Plus 轨外，另实现了一条注解映射轨：

```
@Results(id = "empAnnotationMap", value = {
        @Result(property = "empId",   column = "emp_id", id = true),
        @Result(property = "empName", column = "emp_name"),
        @Result(property = "delFlag", column = "del_flag")
})
@Select("SELECT " + COLUMNS + " FROM emp WHERE emp_id = #{empId}")
Emp selectById(@Param("empId") Integer empId);
```

三条轨（XML / 注解 / MyBatis-Plus）对同一批数据的查询结果逐字段比对完全一致；同时把“唯一不一致的地方”用测试固定下来：emp_id=16 是逻辑删除行，纯 MyBatis 两条轨查得到、MyBatis-Plus 查不到（因为后者自动追加了 del_flag = 0）。

（3）凭据与敏感文件检查。为了不靠人记着“别把口令提交上去”，我把这条要求写成了测试：扫描源码/脚本/SQL 中的明文凭据特征、校验 XML 中的敏感配置必须使用 ${...} 占位符、校验 .gitignore 中的凭据忽略规则处于有效状态。




图 3-4　注入回归与安全扫描测试

## 3.8 任务 6：可运行入口与 IDE 联调

前面几个任务都是通过测试来验证的，但只靠测试不利于演示和排查。所以在 工程里加了一个可直接运行的主类 com.lab.LabApplication，把两个表和各条技术路线串成一次可以看懂的演示：

```
public class LabApplication {
    public static void main(String[] args) throws Exception {
        quietLogging();               // 演示时关掉 SQL 明细，只留干净输出
        checkConnectionAndSchema();   // 1. 连接 + 两张表是否都在
        showDeptTable();              // 2. sys_dept：6 个部门逐行打印
        showEmpTable();               // 3. emp：16 名员工（含 1 条已逻辑删除）
        showConditionQuery();         // 4. 多条件动态 SQL
        showPagination();             // 5. COUNT + LIMIT 分页
        showDeptStatistics();         // 6. JOIN 聚合统计
        showNPlusOneComparison();     // 7. N+1 与 JOIN 的 SQL 条数对照
        showMyBatisPlus();            // 8. BaseMapper 与 @TableLogic
        showWriteAndRollback();       // 9. 增删改后整体回滚，基线不变
        summary();                    // 汇总通过/失败项
    }
}
```

每次演示结束会打印“通过 N 项、失败 0 项”，最后两项专门验证“回滚后库里没有留下演示数据、emp 仍是 16 行”，保证跑十次也不会把库跑脏。

IDEA 里的使用方式：打开工程根目录的 pom.xml（Open as Project）等 Maven 导入完成，然后右键 LabApplication → Run 即可；工程里也预置了运行配置 LabApplication（实验一 Demo）与全部测试（116 个用例），在下拉框里直接选。命令行等价写法：

```
bash scripts/start-lab-mysql.sh                 # 启动实验库（127.0.0.1:3307）
mysql -u root -p < sql/01_schema.sql            # 建库 + emp / sys_dept 两张表
mysql -u root -p < sql/02_seed.sql              # 导入种子数据
mvn exec:java                                   # 运行演示入口
```

连接参数放在 src/main/resources/db.properties（该文件已被 .gitignore 排除，不随工程提交），换机器时只改这个文件即可，不用动代码。

## 3.9 测试与验收

全工程共 15 个测试类 / 116 个测试用例，覆盖环境自检、基础 CRUD、事务、动态 SQL、注入回归、分页、统计与 N+1、MyBatis-Plus 通用 CRUD 与分页、三轨一致性、慢 SQL 插件、安全扫描与脱敏工具。验收方式（在项目根目录执行）：

```
export DB_PASSWORD='<你的数据库口令>'       # 口令只通过环境变量传入，不写入任何文件
bash scripts/run-tests.sh                    # 全量：15 个类 / 116 个用例
mysql -u root -p -e "source sql/03_verify.sql"   # 复核数据基线：16 / 15 / 1 / 12 / 3
```

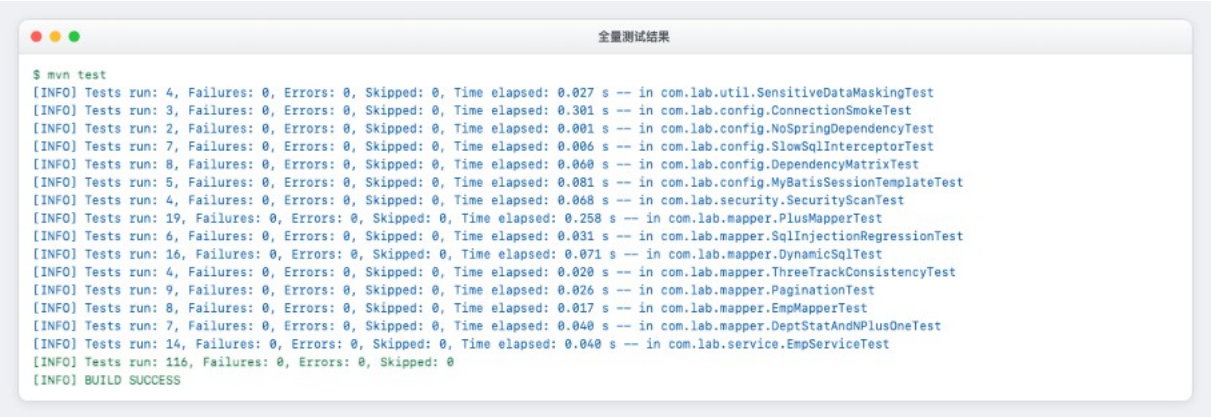



图 3-5　全量测试结果

所有涉及写操作的测试都通过“会话回滚”或“用例结束显式清理”保证数据库基线零污染：测试前后 emp 表始终保持 16 行（未删除 15 行 / 已逻辑删除 1 行 / 在职 12 行 / 离职 3 行）。测试基类还提供 assertBaselineUnchanged()，用于主动发现“测试自己把库写脏了”的情况。

## 3.10 创新点介绍与优劣分析

在完成基础要求之外，我又补充了一些设计。下面按“工程与架构 / 数据与持久层 / 可观测与安全”三类，逐条说明它解决的问题、优势与不足之处。

### （一）工程与架构类

#### ① 单模块 + 分包组织

- 解决的问题：工程要能交给老师一眼看懂，同时两条技术路线不能混在一起。
- 优势：一个 pom.xml 管全部依赖，版本不用跨模块仲裁；用 common / mybatis / plus 三个包把职责分开，找代码时按包名就能定位；构建链路短，IDEA 导入一次即可。
- 不足：所有依赖堆在一个 pom 里，删掉某条技术路线时要手工清理依赖；将来若要拆成独立构件，需要再改成多模块。

#### ② 会话模板（MyBatisSessionTemplate）

- 解决的问题：原生方式下每次写操作都要手写“开会话 → 执行 → 提交/回滚 → 关闭”，容易漏写 commit()，而漏写时 MyBatis 会在关闭会话时静默回滚。

• 优势：事务边界收敛到一处，业务方法只表达“做什么”；异常自动回滚；显著减少重复的样板代码。

• 不足：牺牲了一部分灵活性，特殊场景（多语句事务、批量分批提交）需要额外设计。

**③ 依赖与版本的运行期探针**

• 解决的问题：配置文件里写的版本，不等于运行期真正加载的版本；版本漂移不报错但行为会变。

• 优势：不依赖网络与额外插件；以运行期真实结果为证据（类加载器读版本、JDBC 元数据读驱动、Configuration 读拦截器数量），比配置文件更可信，也能顺手写成断言。

• 不足：只能反映“运行期实际加载的版本”，无法完整替代依赖树分析；命令行与 IDE 的 classpath 若不一致，可能得出误导性结论，需要两边都验证。

**④ 离线测试兜底脚本**

• 解决的问题：某些环境下测试运行器缺少必要组件（surefire 的 JUnit4 provider 是运行时才解析的独立依赖），无法直接执行单元测试。

• 优势：不依赖网络即可跑通全部用例；classpath 显式可控；顺带统一了控制台输出编码，避免中文告警乱码。

• 不足：classpath 是手工拼装的，与 Maven 的真实解析结果可能不一致，需要与构建结果定期对齐。

**（二）数据与持久层类**

**⑤ 表结构增强与语义解耦（status 与 del_flag 分离）**

• 解决的问题：若把逻辑删除标记直接标注在表示业务状态的字段上，“离职”会被当成“已删除”，查询与删除语义整体反转。

• 优势：两个语义维度互不干扰；逻辑删除不污染业务状态；便于按“状态 + 删除标记”建立组合索引；为统计与筛选保留了完整信息。

• 不足：dept 与 dept_id 同时存在，属于有意保留的冗余；逻辑删除与唯一键存在冲突（删除后唯一值仍占位）；手写 SQL 场景下必须自己带上删除条件。

**⑥ 三轨 Mapper 与一致性测试（XML / 注解 / MyBatis-Plus）**

• 解决的问题：同一种查询用不同方式实现，结果是否一致、差异在哪里，往往只能靠人工比对。

• 优势：用断言比对三条轨的结果；把“唯一不一致的地方”（逻辑删除口径）也写进测试，避免以后误判成 bug；为技术选型提供了可量化的依据。

• 不足：同一份查询要维护三套实现，代码量明显增加；一致性测试属于集成级测试，依赖数据库与种子数据，运行成本高于纯单元测试。

**⑦ 双轨分页（自研 PageResult 与框架分页插件）**

• 解决的问题：直接使用框架分页会把“分页是如何实现的”变成黑盒，遇到“总数不对、未加 LIMIT”这类问题时无从下手。

• 优势：两条实现路径互为参照，总数、页数、当前页数据可直接对照；不引入额外组件也能完成分页；SQL 条数与执行过程一目了然。

• 不足：自研分页需要额外执行一次 COUNT 查询，并自行处理页码越界、页大小上限等边界；维护成本高于直接使用现成插件。

**⑧ N+1 现场演示与量化优化**

• 解决的问题：N+1 往往只停留在概念层面，缺少可观察、可复现的现场。

• 优势：用“SQL 条数”把问题量化（7 条 vs 1 条），结论可复现；优化前后结果一致性有断言保证，避免“优化”引入新问题。

• 不足：SQL 条数统计依赖测试期的插件辅助，生产环境需要评估其开销；JOIN 聚合在数据量大时要关注结果集膨胀。

**（三）可观测与安全类**

**⑨ 慢 SQL 拦截器**

• 解决的问题：缺少对“到底哪条 SQL 慢”的可观测性，只能凭感觉。

• 优势：轻量、零额外依赖；拦截点经过实测验证，能够拿到真实 SQL 文本；阈值支持外部参数配置，便于在不同环境调整。

• 不足：目前只做阈值告警，未做耗时指标聚合与上报；逐条计时在高并发下会有微小开销；告警过于频繁时需要配合日志级别调优。

**⑩ 统计查询与执行计划验证**

• 解决的问题：聚合统计的正确性与索引是否真正生效，缺少独立验证。

• 优势：GROUP BY 的统计结果可与逐行求和交叉验证；EXPLAIN 把“索引可用性”变成可检查项，实测命中 idx_emp_status_del 组合索引，避免“以为走索引、其实全表扫描”。

• 不足：小数据量下优化器可能直接选择全表扫描，需要正确理解 possible_keys 与 key 的区别，否则容易误判；统计 SQL 与业务查询分开维护，口径一致性依赖约定。

**⑪ 安全合规闭环**

• 解决的问题：凭据容易被写进代码或配置文件，忽略规则也可能被误改而失效。

• 优势：连接信息全部走环境变量与占位符；敏感文件纳入忽略清单；日志输出做掩码；并把上面的要求写成了检查（源码扫描、忽略规则有效性校验）。

• 不足：扫描基于规则匹配，存在漏报与误报（例如会命中自己的文档示例），需要维护白名单；本地配置文件是否提交仍依赖人的操作习惯；规则需要随项目演进持续更新。

## 四、AI 应用记录

实验过程中我用 AI 编程插件辅助编码和排查。下表是 4 个印象比较深的交互，记录了我对 AI 回答的判断和最后实际怎么改的。

| # | 提示词摘要 | AI 回答要点 | 人工校验 | 我的修正 | 关联提交 |
|---|---|---|---|---|---|
| 1 | “MyBatis-Plus 项目里 BaseMapper.selectById 报 Invalid bound statement (not found)，接口确实继承了 BaseMapper、实体注解也写了，请解释 BaseMapper 的 CRUD 语句是什么时候、由谁注册进 MyBatis 的，并给出不连数据库就能验证‘语句到底注入了没有’的办法。” | ① 指出 BaseMapper 的方法不是注解/XML 声明，而是由 MyBatis-Plus 在配置初始化阶段动态注入 MappedStatement；② 推测是“语句未注册”，建议检查映射器扫描路径与 XML 注册方式（默认了由容器自动扫描的场景）；③ 给出 configuration.hasStatement(...) 的验证思路。 | 部分正确：机制解释方向对，但排查建议默认了“由容器自动扫描装配”的场景（跟这个工程手工装配的方式不符），且把方向带向 mapper 注册 / namespace，与真正的根因（Configuration 类型不对）无关。 | 先用 hasStatement() 实测：用原生 Configuration 注册时 12 条语句一条都没有；改用 MybatisConfiguration + MybatisSqlSessionFactoryBuilder 后全部注入。 | 9fc3d9a → de57fd2 |
| 2 | “实体里 status 表示 1 在职 / 0 离职，我想用 MyBatis-Plus 的 @TableLogic 做逻辑删除，能否直接标在 status 上？如果不可以，请说明会引发哪些具体的 SQL 行为变化。” | ① 给出了该字段上的删除 / 查询 SQL 形态；② 说明会产生与业务状态耦合的问题，建议使用独立字段；③ 但同时也给出了“若坚持复用 status，可通过 @TableLogic(value, delval) 指定值”的做法。 | 部分正确：风险提示正确，但同时也提供了“复用业务字段”的可行写法，容易被误用为推荐方案。 | 实际改成“独立 del_flag 字段”，并用测试验证：误标后 selectCount 从 15 变成 4、deleteById 生成的 SQL 变成 UPDATE emp SET status = 1（把员工改成在职），语义整体反转。 | d6f8ea1 → b501316 |
| 3 | “我给实体加了 Lombok @Builder，之后 MyBatis 查询报 ReflectionException: Error instantiating class ... with invalid types () or values ()., 请解释 MyBatis 创建实体对象的过程，以及 @Builder 为什么会破坏它。” | ① 指出 MyBatis 通过 ObjectFactory 走无参构造反射创建实体；② 说明 @Builder 会生成全参构造、使类不再有隐式无参构造；③ 建议补 @NoArgsConstructor 或移除 @Builder。 | 正确：机制与修复方向都对。 | 采用“移除 @Builder、恢复 @NoArgsConstructor”，与项目约定（实体不使用 @Builder）保持一致；并补充验证：getDeclaredConstructors() 由“参数个数 17”回到“参数个数 0”。 | 13ed0cb |
| 4 | “我在 MyBatis 拦截器里按 SLF4J 的写法写了 LOG.warn("... {} ... {} ...", a, b, c)，编译报‘无法将接口 org.apache.ibatis.logging.Log 中的方法 warn 应用到给定类型’，为什么？” | ① 指出 org.apache.ibatis.logging.Log 没有 SLF4J 风格的占位符重载，只有 warn(String) 与 warn(String, Throwable)；② 建议改为字符串拼接或换成 SLF4J；③ 附带提醒 MyBatis 各类日志实现都走这个统一接口。 | 正确：API 事实准确。 | 改为字符串拼接；同时在 finally 里取 SQL 文本时做了容错（取不到就输出 <unknown sql>），避免“打印日志本身抛异常”掩盖真正的 SQL 异常。 | 8004cf7 |

用 AI 给的代码之前，我会照着下面几条过一遍：

1. 版本一致性：AI 给出的依赖坐标与版本是否真实存在、是否与现有依赖冲突（这次就碰到过被写成不存在的版本号）；

2. 环境假设：AI 是否默认使用了“由容器自动扫描装配”的写法，与这个工程手工装配的方式是否一致；

3. API 真实性：AI 使用的类、方法、重载是否真实存在（如日志接口的占位符重载、拦截器的签名参数类型、BaseMapper 的语句名）；

4. SQL 与映射正确性：AI 写出来的 SQL 列名跟真实表结构对不对得上、有没有用 ${} 拼接用户输入；

5. 是否照搬示例：AI 有时会直接沿用网上示例中的写法（例如把 @TableLogic 标在业务状态字段上），需要结合自己的表设计判断；

6. 测试充分性：AI 生成的测试是否只覆盖“顺利路径”，是否遗漏空值、越界、重复执行等边界；

7. 安全：AI 有没有把明文口令写进代码或配置里，用的密钥格式是不是真的存在。

## 五、问题与解决方案

### 问题 1　MyBatis-Plus 通用 CRUD 报 Invalid bound statement (not found)

现象：EmpPlusMapper 继承了 BaseMapper、实体上 @TableName/@TableId/@TableLogic 都写了，但调用 selectById 时抛：

```
org.apache.ibatis.binding.BindingException: Invalid bound statement (not found):
    com.lab.plus.mapper.EmpPlusMapper.selectById
```

分析与排查（关键是用“不连数据库的判据”把猜测变成事实）：

① 第一层：先确认语句到底有没有被注入。执行 configuration.hasStatement("com.lab.plus.mapper.EmpPlusMapper.selectById")，结果为 false —— 说明不是 SQL 写错，而是语句根本没注册。

② 第二层（真正原因）：装配时用了原生 org.apache.ibatis.session.Configuration。BaseMapper 的通用 CRUD 是由 MyBatis-Plus 在配置初始化阶段注入的，只有其子类 MybatisConfiguration 才会触发这套注入逻辑；配置类用错，接口继承与实体注解看起来都“对”，但语句一条也没有。

解决：改用 MybatisConfiguration + MybatisSqlSessionFactoryBuilder：

```
// 修复前：原生 Configuration，12 条通用语句一条都没注入
// Configuration configuration = new Configuration();


// 修复后：MyBatis-Plus 自己的 Configuration
MybatisConfiguration configuration = new MybatisConfiguration();
SqlSessionFactory factory = new
MybatisSqlSessionFactoryBuilder().build(configuration);
```

验证方式：hasStatement() 对 insert / delete / deleteById / deleteBatchIds / update / updateById / selectById / selectBatchIds / selectList / selectMaps / selectObjs / selectCount 共 12 条全部为 true，并把这条断言写进了 DependencyMatrixTest。

### 问题 2　测试里 SQL 计数恒为 0，而 SQL 日志显示语句确实执行了

现象：为了量化 N+1 问题，写了一个统计 SQL 条数的 MyBatis 插件，但计数始终为 0（expected:<7> but was:<0>），同时 SQL 日志显示 7 条 SQL 都执行了。

分析与排查：

① 第一层：最初想在用例结束时把插件从配置中移除，报 java.lang.UnsupportedOperationException ... UnmodifiableCollection.remove —— Configuration.getInterceptors() 返回的是不可变列表，不能 remove。于是改为“只在 @BeforeClass 注册一次、每次测量前清零”的幂等注册方式。

② 第二层（计数为 0 的原因）：拦截点选错了。最初拦的是 Executor#query，但 DefaultSqlSession 调用的是 4 参重载，内部再调用 6 参重载，属于对象内部调用、不经过代理；嵌套查询由 DefaultResultSetHandler 直接持有 Executor 执行，同样不经过代理。因此无论只拦 4 参还是只拦 6 参，都统计不到（或统计不全）。

③ 第三层（调试中新发现的一层）：替身被“双重包装”。构造 StatementHandler 替身时已经调用过一次 Plugin.wrap，测试里又用 interceptor.plugin() 包了一层，结果拦截器落在内层，外层 PluginWrapper 的签名表匹配不到方法，直接 method.invoke 透传，计数恒为 0 —— 现象与“插件根本无效”完全一样，极具迷惑性。

解决：统计 SQL 条数改用 StatementHandler#prepare —— 每次 SQL 执行都会经 Configuration.newStatementHandler() 新建 handler 并被全局拦截器链包装，主查询与嵌套子查询都会被计入；同时把“只包装一次”写成注释固定在测试里。

验证方式：JOIN 场景 1 条、N+1 场景 7 条，与 SQL 日志逐条对应。




图 5-1　N+1 与 JOIN 两种写法的 SQL 条数对照

结论：这条经验后来直接用在慢 SQL 插件上 —— 要统计 SQL 条数可拦 StatementHandler#prepare；要统计执行耗时则必须拦 query/update（prepare 只做“准备语句”）。

### 问题 3　@TableLogic 标错字段导致查询与删除语义整体反转

现象：最初把 @TableLogic 标在了表示业务状态的 status 字段上（1 在职 / 0 离职）。编译通过、运行不报错，但：

```
selectCount(null)                          // 结果 4，而不是预期的 15
selectList(...)                            // 只返回 4 行，且全是离职员工
deleteById(id)                             // 生成的 SQL 变成 UPDATE emp SET status = 1
```

分析：MyBatis-Plus 会把 @TableLogic 的 value 当作“正常值”、delval 当作“已删除值”。标在 status 上之后，框架认为只有 status = 1 的记录是“未删除”，于是把“离职（0）”过滤掉，删除操作也变成了“把员工改成在职”。本质是“业务状态”与“逻辑删除”两个语义维度被强行合并到一个字段。

解决：迁移到独立的 del_flag 字段，status 只表示在职 / 离职：

```
/** 业务状态：1 在职 / 0 离职 */
@TableField("status")
```

```
private Integer status;


/** 逻辑删除标记：0 正常 / 1 已删除 —— 只有这个字段可以承载 @TableLogic */
@TableLogic(value = "0", delval = "1")
@TableField("del_flag")
private Integer delFlag;
```

验证方式：selectCount 恢复为 15；deleteById 生成 UPDATE emp SET del_flag = 1 且物理行仍在（用纯 MyBatis 会话复核 del_flag=1、status 不变）；三轨一致性测试把这个口径差异固定下来。

**问题 4　分页功能静默失效：total 恒为 0 且未追加 LIMIT**

现象：调试其它问题时临时把 MyBatis-Plus 的分页拦截器注册注释掉了，之后忘记恢复。结果 selectPage 不报任何异常，但：

```
[PLUS/PAGE] MP selectPage -> total=0, pages=0, records=15, ids=[1, 2, ..., 15]
java.lang.AssertionError: 总数应为 15  预期:15 实际:0
```

分析：total 与 LIMIT 都是由分页插件在查询过程中设置的；插件未注册时，selectPage 退化成“执行一次普通查询并返回全部数据”，total 保持初始值 0。因此这是一个没有异常、只有断言才能发现的静默失效。

解决：恢复插件注册，并把“不连库的判据”写成断言 —— PlusConfigurationFactory.buildConfiguration().getInterceptors().size() 由 1 变成 0 即可判定失效。日志随即恢复为“先 SELECT COUNT(*) AS total、再 ... ORDER BY emp_id ASC LIMIT ?”两条语句，total=15、pages=4。

体会：功能“不报错”不等于“正确”。对这类配置型功能，最可靠的办法是把“期望行为”写成断言（本例断言了 total、pages 与当前页数据），而不是只在页面上“看起来有数据”。

# 六、实验总结与思考

## 6.1 思考题解答

**（1）简述 #{} 与 ${} 的区别，并说明为什么禁止使用 ${} 接收用户输入。**

#{} 是 PreparedStatement 的预编译占位符，MyBatis 会把它编译成 ?，参数值通过 setXxx() 绑定，天然免疫 SQL 注入，且能复用执行计划；${} 是纯字符串替换，编译期就把值拼进 SQL 文本，既可能破坏 SQL 结构，也会造成注入。用户输入会改变 SQL 结构，例如输入 ' or '1'='1，可能拼成：

```
select * from emp where emp_name = '' or '1'='1'
```

从而绕过条件、泄露全部数据；若用于 order by ${sort} 也可能注入恶意 SQL。因此用户输入应使用 #{}；必须用 ${} 时只能用于可控字段，并做枚举/白名单校验（EmpQuery#getSafeOrderBy() 就是按这个思路写的）。

**（2）简述 SqlSessionFactory、SqlSession 与 Mapper 接口三者的关系及生命周期。**

| 对象 | 作用 | 生命周期 |
| --- | --- | --- |
| SqlSessionFactory | 读取 MyBatis 配置，创建 SqlSession | 应用级，通常单例，线程安全，应用启动后长期持有 |
| SqlSession | 代表一次数据库会话，执行 SQL、获取 Mapper、提交/回滚事务 | 请求级或方法级，非线程安全，不能共享；用完必须 close() |
| Mapper 接口 | MyBatis 通过 JDK 动态代理生成实现类，绑定 XML/注解中的 SQL | 原生 MyBatis 中通常与 SqlSession 同生命周期，方法级使用；Spring 集成中 Mapper Bean 可单例注入，但底层 SqlSession 是线程绑定、请求级的 |

实测补充：每次 sqlSession.getMapper(EmpMapper.class) 都会返回一个新的 JDK 动态代理实例，但它们共享同一个会话，因此看到的是同一份数据（EmpMapperTest.t8 对此有断言）。

**（3）什么是 N+1 查询问题？结合这次实验的 emp 表，举出一个会产生 N+1 问题的查询场景并给出优化思路。**

N+1 指的是：本可以用 1 条 SQL 完成的事，被拆成“1 条主查询 + N 条子查询”，数据库往返次数随数据量线性增长。这次实验的场景是“查询每个部门及其员工”：先查 6 个部门（1 条 SQL），再对每个部门分别查一次员工（6 条 SQL），合计 7 条。项目里用两种写法做了对照：

- N+1 版：<collection> 上使用 select="...selectEmpsByDeptId"（嵌套子查询）→ 实测 7 条 SQL；
- 优化版：用 LEFT JOIN 一次取出部门与员工，再通过 <collection columnPrefix="e_"> 在内存中聚合 → 实测 1 条 SQL。

两者返回的数据完全一致（有测试断言逐字段相等），差别只在 SQL 条数。为了把结论量化，我用一个 MyBatis 插件统计 SQL 条数并写成断言，而不是只看日志“感觉慢”。优化思路即：能用 JOIN 聚合的尽量一条 SQL 取完；如果必须多次查询，则改为一次批量查询（IN + 内存分组）。

**（4）对比手写 XML 动态 SQL 与 MyBatis-Plus 条件构造器两种实现方式，说明各自的适用场景。**

| 维度 | XML 动态 SQL | MyBatis-Plus 条件构造器 |
| --- | --- | --- |
| 可读性 | 复杂条件、分支、复用片段（<sql>）表达力强 | 简单的等值/范围/排序条件极其简洁，Lambda 写法能借助方法引用避免列名字符串写错 |
| 动态能力 | <if>/<where>/<trim>/<set>/<choose>/<foreach> 完整覆盖 | 靠 condition 重载判断是否拼接，分支逻辑较弱 |
| 多表/复杂 SQL | 支持任意 SQL（JOIN、聚合、子查询、窗口函数） | 主要面向单表；复杂 SQL 仍要写 XML 或 @Select |
| 动态列名 | 可用 ${} + 白名单 | 排序等可用 orderBy 相关 API |
| 维护成本 | SQL 集中在 XML，便于 DBA 审阅 | 单表 CRUD 几乎零 SQL，开发效率高 |

**（5）结合本次实验，谈谈 AI 生成代码可能出现哪些质量问题，你采取了哪些人工校验手段。**

我在实验中实际遇到的 AI 代码质量问题主要有五类：

1. 环境假设错位：这个工程是手工装配，但 AI 给出的排查/示例代码默认是“由容器自动扫描装配”的写法，直接照抄会引入与项目装配方式不符的配置；

2. API 记忆混淆：例如按 SLF4J 风格写 MyBatis 的日志调用 LOG.warn("... {} ...", a, b)，而 org.apache.ibatis.logging.Log 并没有占位符重载，直接编译失败；

3. 照搬示例导致语义错误：AI 生成的实体注解可能把 @TableLogic 直接标在业务状态字段上，编译与运行都不报错，但会让“离职”被当成“已删除”，查询与删除语义整体反转；

4. 版本幻觉与依赖冲突：AI 给出的依赖版本可能不存在或与现有依赖冲突，造成依赖解析失败或运行期版本漂移；

5. 给出的方法名与实际不符：例如想当然地把 BaseMapper 的注入语句写成 selectOne / selectMapsPage，实际并不存在（实测出来的 12 条注入语句名单跟我的印象不一样，断言立刻失败）。

对应的校验手段：依赖坐标与版本必须能在本地仓库解析且版本唯一；AI 给出的 API 必须先编译通过再用；涉及框架语义（如逻辑删除、主键策略、插件注册）的改动必须落到断言上；所有 AI 生成的 SQL 都要与真实表结构逐字段核对；写操作必须确认事务边界；安全方面（明文凭据、注入风险）单列一条检查。

## 6.2 实验收获

技术方面：把 MyBatis 从配置到执行的链路走通了一遍（配置 → SqlSessionFactory → SqlSession → Mapper 代理 → SQL → 结果映射），能熟练使用动态 SQL 的六类标签；理解了 #{}

与 ${} 的编译期差异并用实验证实了注入风险；掌握了 MyBatis-Plus 的通用 CRUD、条件构造器、分页插件与逻辑删除，并清楚了“它在背后替我做了什么”。

工程方面：学会了用分包把两条技术路线分开、用一个 pom 统一版本、用会话模板收敛事务边界、用测试基类保证数据基线零污染；也第一次体会到“测试基础设施本身也会出错”——比如双重包装会让插件测试得出假结论、用旧会话验证提交结果会得到假失败、拦截器清理需要操作原始列表而不是不可变视图。

与 AI 配合方面：最大的收获是认识到 AI 擅长“改得动语法”，但不一定“找得到根因”。同一句 Invalid bound statement 报错，背后可能是映射文件位置、资源未打包、或框架配置类用错三种完全不同的原因；AI 往往会给出“看起来最像”的那一种。要判断它是否正确，靠的不是再问一遍 AI，而是自己设计一个不依赖数据库的判据（例如 configuration.hasStatement(...)、getDeclaredConstructors()、getInterceptors().size()）去证实或证伪。

安全方面：以前只记得“口令别提交”，但没人验证过到底做没做到。这次连接信息全部走环境变量、敏感文件加进 .gitignore、日志输出做掩码，还写了测试去检查源码里没有明文凭据、忽略规则是生效的。

思政体会：整个实验最深的感受是“严谨”二字的分量。有几类缺陷（版本漂移、插件失效、逻辑删除语义反转、隔离级别导致的假失败）不报任何错误，只会在某次 git add . 或某次线上查询中暴露；而排查它们往往需要回到“机制”层面，而不是停留在“让报错消失”。这让我理解了“工匠精神”在软件工程里的具体含义：不仅要让程序跑通，还要让预期的行为被测试固定下来。

## 6.3 局限与改进方向

1. 测试运行方式：某些环境下 Maven 可能缺少 surefire 的 JUnit4 provider，目前同时提供离线兜底脚本；联网预热本地仓库后可直接使用 mvn test，并接入 CI；

2. 测试数据依赖基线：现有测试依赖固定的种子数据（16 行），若将来要并行执行，需要改造为“每个用例自建独立数据集 + 自清理”；

3. 逻辑删除与唯一键的冲突：emp_no、email 是唯一键，逻辑删除后这些值仍占位，无法直接复用。后续可考虑“逻辑删除 + 唯一键包含 del_flag”或“删除时对唯一字段追加时间戳后缀”等方案；

4. 慢 SQL 插件的能力边界：目前只做阈值告警输出，未做耗时指标聚合与上报，后续可接入监控；

5. 索引验证范围：EXPLAIN 目前只覆盖主要的几个查询，随着查询条件增多，需要把执行计划检查纳入回归测试；

6. 测试基础设施仍需加固：SQL 计数插件依赖反射摘除，属于测试期的临时手段，更稳妥的做法是把计数能力做成可插拔的测试组件。

# 附录 A：开发过程中遇到的典型报错索引

下表汇总了本次实验编码过程中出现并解决的典型问题：

| # | 报错现象（关键行） | 根本原因 | 解决方式 |
|---|---|---|---|
| 1 | 依赖无法解析：Could not find artifact com.mysql:mysql-connector-j:jar:9.9.9 | 依赖版本号写错（写成了不存在的版本） | 核对本地仓库真实可用版本，改回 9.0.0 并由父 pom 统一管理 |
| 2 | LogException: Error setting Log implementation. Cause: NoClassDefFoundError: org/apache/log4j/Priority | 配置里声明了 logImpl=LOG4J，但依赖中没有 log4j | 补回 log4j:log4j:1.2.17 依赖 |
| 3 | ReflectionException: There is no getter for property named 'empname' | 映射文件中参数占位符大小写写错（#{empname}） | 改为 #{empName}，与实体属性名一致 |
| 4 | 查询无异常，但 empName 字段为 null | <resultMap> 中列名拼写错误（emp_namee），映射静默失败 | 修正列名；同时用断言而非“看日志”来验证映射完整性 |
| 5 | 插入返回 1 行，但实体主键为 null | <insert> 缺少 useGeneratedKeys / keyProperty | 补上两个属性以回填自增主键 |
| 6 | 程序提示新增成功，重新连接后数据消失 | 写操作后未提交事务，会话关闭时被回滚 | 统一改为会话模板自动提交；测试改为“用新会话复核” |
| 7 | BindingException: Parameter 'q' not found. Available parameters are [arg0, arg1, param1, param2] | 多参数方法未用 @Param 显式命名 | 补 @Param("q") / @Param("page") |
| 8 | 模糊查询输入 ' OR '1'='1 返回全表 | 误用 ${} 拼接用户输入 | 改为 #{}（预编译占位符），并补注入回归测试 |
| 9 | Invalid bound statement (not found): ...EmpPlusMapper.selectById | 装配 MyBatis-Plus 时使用了原生 Configuration，通用 CRUD 语句未被注入 | 改用 MybatisConfiguration + MybatisSqlSessionFactoryBuilder |
| 10 | 查询只返回 4 行且全是离职员工；deleteById 变成 SET status = 1 | @TableLogic 被标注在业务状态字段 status 上 | 迁移到独立的 del_flag 字段，status 只表示在职 / 离职 |
| 11 | 断言失败：mybatis 预期 3.5.16 实际 3.5.15 | 子模块显式声明了 mybatis 版本，覆盖了父 pom 的统一管理 | 删除子模块的显式版本，交回父 pom 仲裁 |
| 12 | 分页 total=0、SQL 未追加 LIMIT，且无任何异常 | 分页插件未注册到拦截器链 | 恢复 MybatisPlusInterceptor 注册；用不连库的判据核对拦截器数量 |
| 13 | 凭据文件出现在 git status 待提交列表中 | .gitignore 中的忽略规则被注释掉后未恢复 | 恢复规则；并补安全扫描测试校验“规则必须有效” |
| 14 | 编译报“无法将构造器 Emp 应用到给定类型”；运行期 ReflectionException: Error instantiating class | 实体加了 Lombok @Builder 且去掉了无参构造，MyBatis 反射创建实体失败 | 移除 @Builder、恢复 @NoArgsConstructor |
| 15 | 慢 SQL 测试计数恒为 0，而 SQL 日志显示 7 条语句都执行了 | StatementHandler 替身被 Plugin.wrap 双重包装，拦截器落在内层导致签名不匹配 | 只包装一次；统计条数改拦 prepare，统计耗时拦 query/update |
| 16 | DELETE 语句 10 秒后超时：MySQLTimeoutException: Statement cancelled due to timeout | 本会话有未提交的 UPDATE 持有行锁，另一个连接去清理同一行被阻塞 | 先在 finally 中 rollback / commit 释放锁，再做跨连接清理 |

## 附录 B：Git 提交记录

代码分两阶段提交：先搭骨架和 SQL，再补业务、测试与文档。每条都写了改了什么，便于按时间回溯。共 17 次提交，覆盖 2 个日期。

| 序号 | 提交时间 | Commit | 提交内容 |
|---|---|---|---|
| 1 | 2026-10-11 09:30 | b0d2537 | chore: 初始化 emp-mybatis 工程（.gitignore / .gitattributes / Maven Wrapper） |
| 2 | 2026-10-11 09:40 | 42b00b4 | docs: 补 README 与协作说明 |
| 3 | 2026-10-11 10:05 | a0a045f | build(pom): 单模块 pom，统一管理 MyBatis / MyBatis-Plus / MySQL 驱动版本 |
| 4 | 2026-10-11 10:40 | 0d9d1f7 | feat(sql): 新增 emp / sys_dept 建表脚本与种子数据 |
| 5 | 2026-10-11 14:20 | e76193e | feat(common): 新增实体、DTO、工具类与异常体系 |
| 6 | 2026-10-11 18:10 | 17141c0 | feat(mybatis): 手工装配 SqlSessionFactory，配置只保留占位符 |
| 7 | 2026-10-11 20:15 | 248b23e | feat(mapper): 新增 XML 与注解两条 Mapper 轨及映射文件 |
| 8 | 2026-10-12 11:35 | 2aecad3 | feat(service): 新增 EmpService，事务边界交给会话模板 |
| 9 | 2026-10-12 14:25 | 5a61dd8 | feat(plus): 手工装配 MybatisConfiguration 与 MyBatis-Plus 拦截器 |
| 10 | 2026-10-12 17:40 | bbdef3c | feat(plugin): 新增慢 SQL 拦截器与日志配置 |
| 11 | 2026-10-12 20:30 | 31cfe8d | feat(app): 新增可运行入口 LabApplication，串起两个表的完整演示 |
| 12 | 2026-10-12 21:10 | bdad329 | test: 新增 15 个测试类共 116 个用例 |
| 13 | 2026-10-12 22:30 | 2327115 | docs: 补充 AI 代码甄别记录 |
| 14 | 2026-10-12 23:20 | 4871d09 | docs: 新增答辩演示脚本、演示手册与 IDEA 运行配置 |
| 15 | 2026-10-12 23:30 | 19358f3 | docs: 新增答辩速查卡 |
| 16 | 2026-10-12 23:35 | 574ad0c | docs: 修正手册中的仓库地址 |
| 17 | 2026-10-12 23:40 | db039f0 | chore(scripts): 新增工程自带的 MySQL 启停脚本，数据目录随工程走 |

提交时间跨度：2026-10-11 09:30 至 2026-10-12 23:40。

按日期统计：

| 日期 | 提交次数 | 主要工作 |
|---|---|---|
| 2026-10-11 | 7 | 工程初始化、单模块 pom、建表脚本与种子数据、实体 DTO 工具类、SqlSessionFactory 手工装配、XML 与注解 Mapper |
| 2026-10-12 | 10 | Service 与事务模板、MyBatis-Plus 装配、慢 SQL 插件与日志、可运行入口与 116 个测试用例、文档与脚本 |